Memória virtual

Universidade Estadual de Mato Grosso do Sul
Bacharelado em Ciência da Computação
Sistemas Operacionais
Prof. Fabrício Sérgio de Paula

Tópicos

- Antecedentes
- Paginação por demanda
- Cópia após gravação
- Substituição de páginas
- Alocação de quadros
- Atividade improdutiva
- Arquivos mapeados em memória

Antecedentes

- Capítulo anterior: alocação contígua, paginação, segmentação
  - Processo inteiro na memória física
  - Limita tamanho máximo de um programa: memória física
    - Overlays e carga dinâmica podem minimizar problema
- Análise de programas reais: programa todo na memória não é necessário
  - Existem procedimentos (ex.: para tratar erros) e dados que raramente são usados
    - Há alocação de memória além do necessário para dados também

Antecedentes

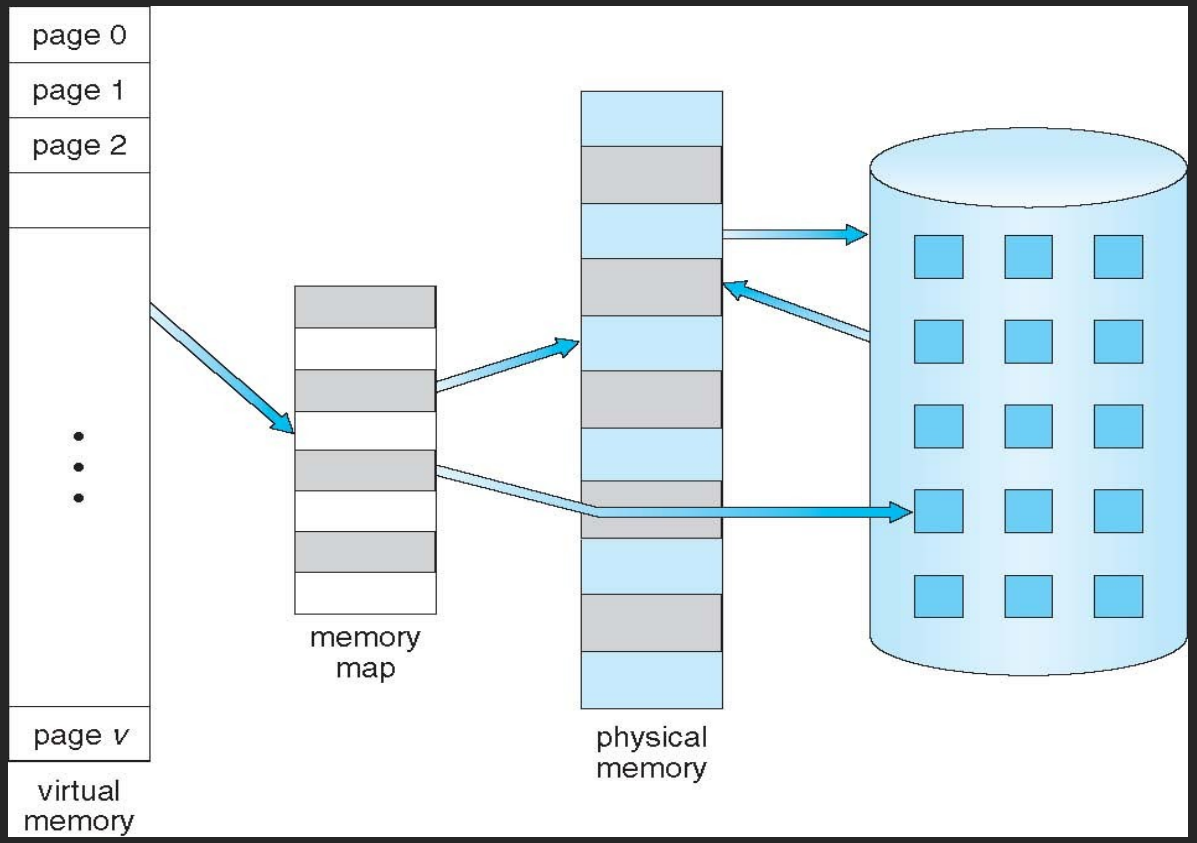
- Benefícios de não carregar programa inteiro na MP:
  - Não limita tamanho do programa ao tamanho MP
    - Espaço de endereçamento virtual maior que físico
  - Cada programa individualmente ocupa menos memória
    - Podem ser carregados mais programas: aumenta utilização da CPU e throughput, sem aumentar tempo de resposta e turnaround
  - Menos E/S é necessária para carregar cada programa e dar início (somente carrega parte dele)

Antecedentes

- Memória virtual separa memória lógica (usuário) de memória física
  - Programador conta com memória (lógica) extensa
    - “Não precisa” se preocupar com quantidade disponível
  - Uso de páginas compartilhadas
    - Permite compartilhar dados na memória e arquivos
    - Aumenta o desempenho na criação de novas cópias de processos
  - Implementação de memória virtual
    - Em geral através de paginação por demanda (ou segmentação)

Antecedentes

- Ilustração de memória virtual maior que física:

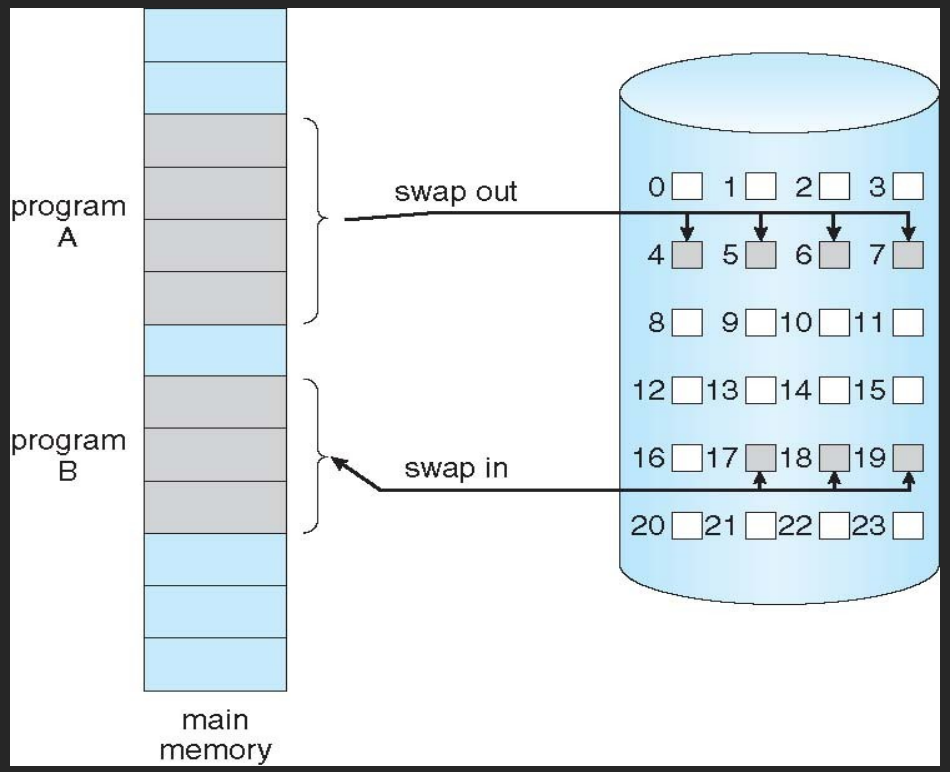


Paginação por demanda

- Paginação por demanda: sistema de permuta
  - Processos residem no disco
  - Quando se deseja executar processo: conduz à memória (swap in)
    - Só carrega a página necessária na memória: à medida que precisar, carrega outras
  - Eventualmente é necessário retirar páginas da memória (swap out)
    - Páginas podem ser do mesmo processo ou de outros

Paginação por demanda

- Transferência de memória paginada para espaço em disco contíguo:

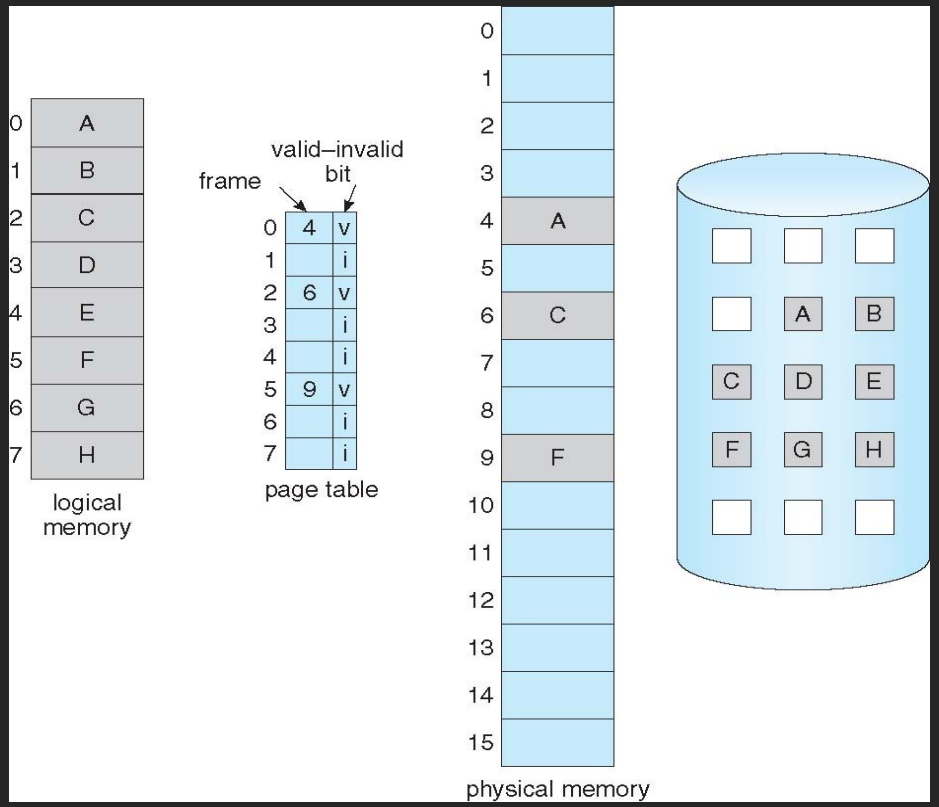


Paginação por demanda

- Paginador: responsável por movimentar as páginas
  - Durante a execução de programa, paginador precisa saber as páginas necessárias para executar
    - Evita de ler as páginas desnecessárias: ganho em tempo e espaço
- É necessário distinguir páginas que estão no disco e que estão na memória
  - Usa-se hardware especial: bit válido-inválido
    - Bit com valor válido: indica que página está na memória
    - Valor inválido: página não é válida (não está no espaço de end.) ou é válida mas não está na memória (tabela pode conter end. no disco)

Paginação por demanda

- Situação da tabela de páginas quando algumas páginas não estão na memória:



Paginação por demanda

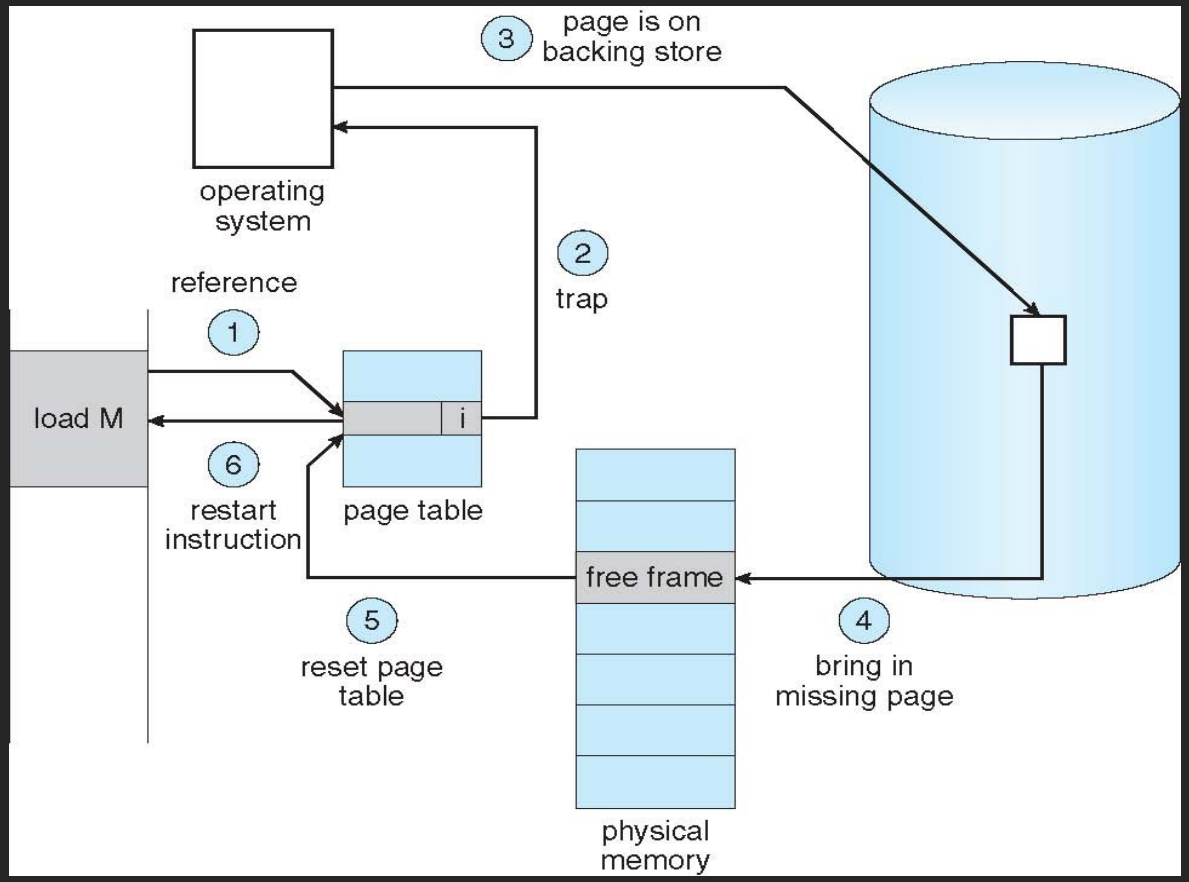
- Se processo não tenta acessar página inválida, execução prossegue normalmente
  - Caso contrário, tentativa de acesso gera exceção de erro de página (ou falta de página): hardware detecta e SO trata
    - Essa exceção não é erro do programa
      - Endereço acessado está dentro do espaço de endereçamento do processo
    - Exceção é “falha” do SO, que deveria ter carregado página antes

Paginação por demanda

- Conceitos básicos (cont.):
  - Tratamento da exceção erro de página:
    1. Verifica uma tabela interna (em geral no BCP do processo) para determinar se referência à memória foi válida ou inválida
    2. Se referência foi inválida (tentativa de acesso fora do espaço de end.), processo é terminado. Se foi válida, é iniciado o carregamento da página
    3. Aloca um quadro livre: através da lista de quadros livres
    4. Inicia leitura do disco para cópia no quadro alocado
    5. Ao término, altera a tabela de páginas, indicando o endereço do novo quadro e marcando a página como válida
    6. Reinicia a instrução que foi interrompida pela exceção

Paginação por demanda

- Passos na manipulação de falta de página:



Paginação por demanda

- Como reiniciar a execução do processo???
  - Contexto é salvo antes de tratar a exceção erro de página
- Dessa forma é possível executar processo com apenas parte na MP
  - Sempre que ocorre acesso a posição fora da memória alocada:
    - Hardware gera exceção para SO
    - SO carrega página necessária
    - Execução do processo é reiniciada

Paginação por demanda

- Paginação por demanda pura: nunca página é conduzida à memória, a menos que seja solicitada
  - Permite iniciar a execução de um processo sem páginas na memória
    - SO posiciona o PC para a primeira instrução do processo:
      - A página não estará residente na memória
      - Ocorre a exceção e o SO carrega apenas a página necessária: processo continua
      - Outras exceções ocorrerão durante a execução e as páginas só serão carregadas nessas situações

Paginação por demanda

- Se cada instrução faz referência a mais de um endereço de memória:
  - Ocorrem múltiplos erros de páginas por instrução executada
  - Desempenho fica degradado: inaceitável
- Entretanto, análise da execução de processos demonstram:
  - Esse comportamento é muito improvável
  - Em geral: programas possuem localidade de referência
    - É provável que as próximas referências estão “por perto”
    - Desempenho da paginação por demanda fica razoável

Paginação por demanda

- Hardware necessário para paginação por demanda:
  - Tabela de páginas: bit válido-inválido
  - Memória secundária: disco rígido
    - Responsável por manter as páginas que não estão na MP
  - Capacidade de reiniciar a execução de instrução
    - Instruções complexas (ex.: movimentação de bytes com sobreposição), em algumas arquiteturas

Paginação por demanda

- Desempenho da paginação por demanda:
  - Paginação por demanda pode afetar muito o desempenho
  - Impacto pode ser calculado através do tempo de acesso efetivo à memória paginada
  - Tempo de acesso à memória (ma) oscila de 10 a 200ns (MP)
    - Esse é o tempo quando não há erros de página
    - Caso haja:
      - É necessário ler do disco a página para depois fazer acesso
  - $0 \leq p \leq 1$ é a probabilidade de erro de página
  - Tempo de acesso efetivo =

$$(1-p) * ma + p * (\text{tempo gasto pelo erro de página})$$

Paginação por demanda

- Cálculo do tempo gasto pelo tratamento do erro de página inclui:
  1. Exceção para SO;
  2. Salvar contexto do processo;
  3. Determinar que interrupção foi erro de página;
  4. Verificar se referência foi válida e determinar página no disco;
  5. Realizar leitura do disco para quadro livre:
     (a) Espera em fila de disco;
     (b) Aguarda pela busca do dado no disco (tempo de latência);
     (c) Aguarda a transferência da página para o quadro;
  6. Enquanto espera, aloca CPU a outro processo;
  7. Gera interrupção do disco;
  8. Salva contexto do processo atual;
  9. Determina que a interrupção veio do disco;
  10. Corrige a tabela de página;
  11. Espera nova alocação da CPU para o processo que sofreu a exceção;
  12. Restaura contexto e retoma execução.

Paginação por demanda

- Tempo gasto pelo tratamento do erro é dominado por três tarefas:
  1. Atendimento à interrupção por erro de página
  2. Leitura da página
  3. Reinício do processo
- Tarefas 1 e 3: centenas de instruções cada uma (código bem feito)
  - 1 a 100 µs cada
- Passo 2: próximo a 8 ms (sem espera na fila)
  - Disco típico: busca 5ms + latência 3ms + transferência 0.05ms
- Tarefas 1 + 2 + 3: aproximadamente 8ms

Paginação por demanda

- Tempo de acesso efetivo:
  - Considerando ma = 200 ns
  - Tempo gasto pelo erro de página = 8 ms

    = (1-p)*200ns + p*8ms

    = (1-p)*200ns + p*8.000.000ns

    = 200 + 7.999.800*p
- Tempo de acesso efetivo é diretamente proporcional à taxa de erros de página

Paginação por demanda

- Para garantir uma degradação máxima de 10%, tempo médio vai de 200ns para, no máximo, 220ns

$$200 + 7.999.800*p < 220$$

$$7.999.800*p < 20$$

$$p < 1/(399.990)$$

- Isso significa que deve ocorrer menos que 1 erro de página a cada 399.990 acessos à memória!

Paginação por demanda

- O desempenho da paginação por demanda pode ser melhorado:
  - Fazendo acesso ao disco de uma forma mais rápida
    - Utilizar espaço de troca (partição swap, arquivo pagefile)
      - Não usa estrutura de arquivos/diretórios
      - Alocação e localização mais rápida
      - Resulta em acesso mais rápido que sistemas de arquivos comuns

Cópia após gravação

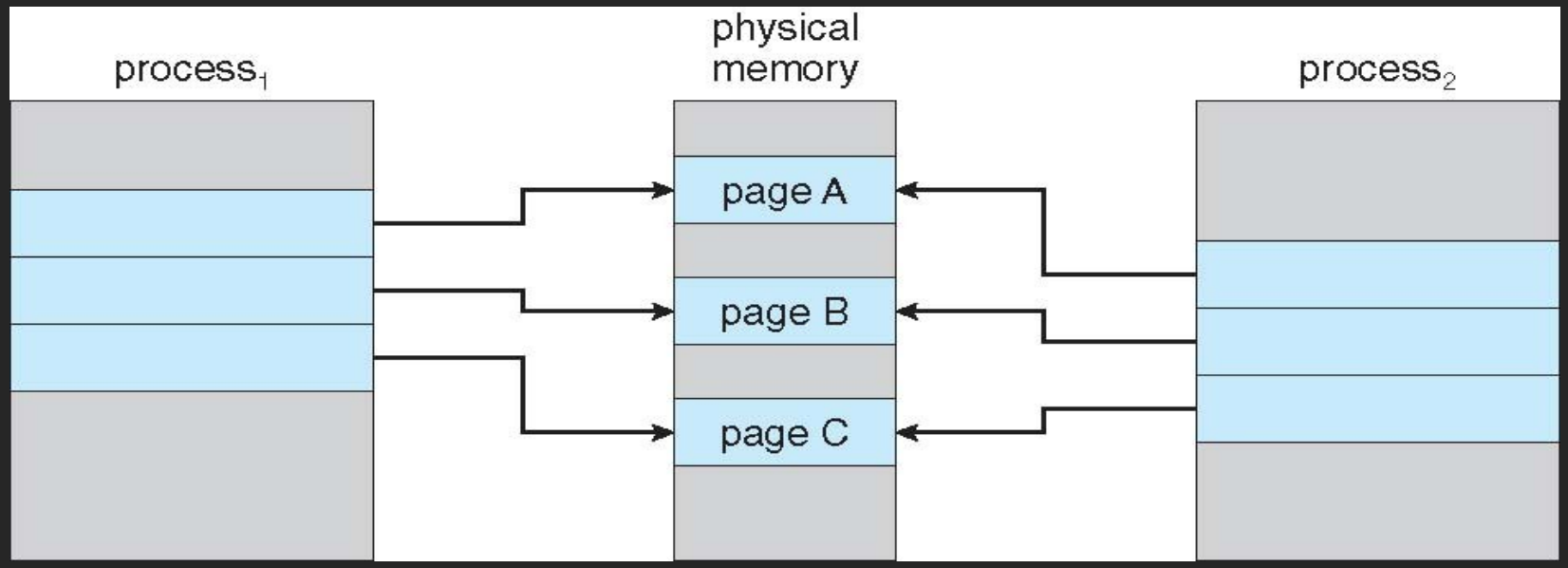
- Cópia após gravação: técnica que permite pai e filho compartilharem inicialmente as páginas
  - Páginas compartilhadas são marcadas com tipo cópia-após-gravação
  - Se processo tentar gravar em página compartilhada: será criada para ele uma cópia
  - Páginas não modificadas continuam compartilhadas
- Cópia após gravação torna criação de processos mais eficiente
  - Mecanismo usado pelo Windows XP, Linux e Solaris

Cópia após gravação

- Alocação de página:
  - SOs normalmente criam cadeia de páginas livres
    - Páginas são preenchidas com zeros: apagam conteúdo anterior
    - Quando um processo precisa ser expandido, SO aloca página da cadeia
  - Mecanismo de cópia-após-gravação
    - Nova página (cópia) será feita em página com zeros
- Criação de processos:
  - Com fork(): faz cópia após gravação
  - Com vfork(): não faz cópia após gravação
    - Processo pai é suspenso
    - Processo filho usa espaço de endereçamento do pai
      - Se alterar páginas, o pai terá conteúdo alterado
    - Recomenda-se usar somente se filho faz exec() logo após vfork()

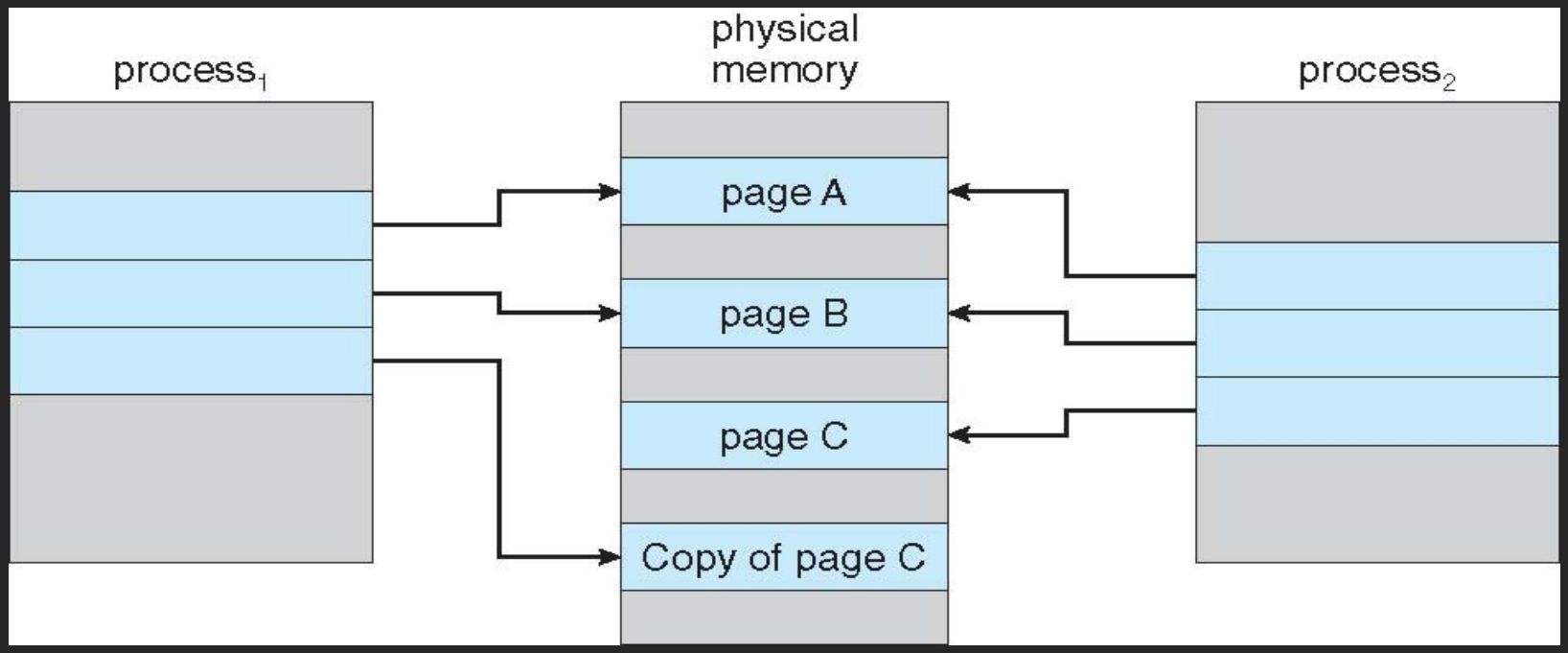
Cópia após gravação

- P1 e P2 compartilhando páginas A, B e C:



Cópia após gravação

- Situação após P1 modificar página C:

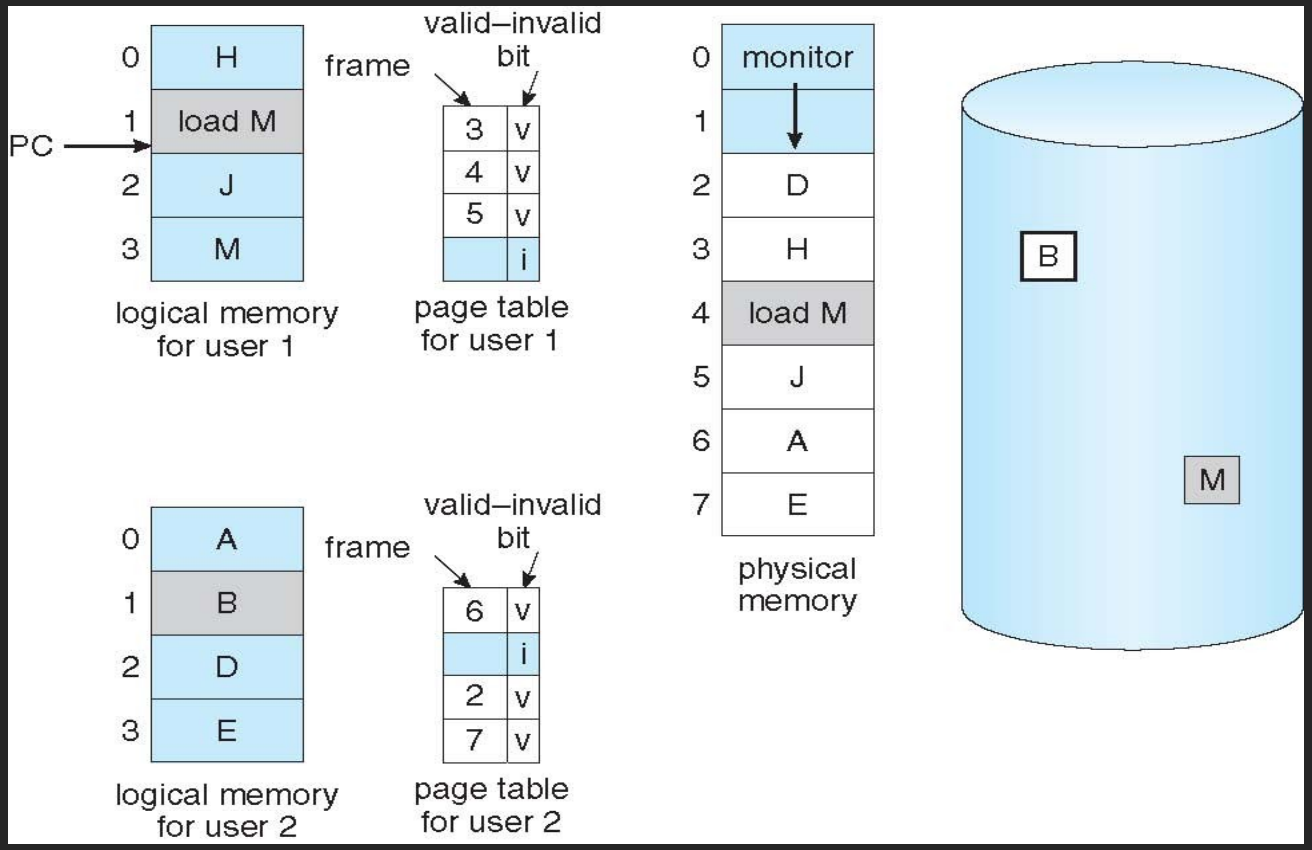


Substituição de páginas

- Memória virtual: espaço virtual muito grande
  - Compensa ficar aumentando muito grau de multiprogramação?
  - Até que ponto utilizar arquivos mapeados em memória?
- Superalocação de memória: processo necessita carga de nova página, mas não há quadro disponível
  - O que fazer?
    - Terminar o processo solicitante: opção ruim
    - Retirar um processo inteiro da memória: pode ser boa
    - Fazer substituição de página

Substituição de páginas

- Exemplo de superalocação:

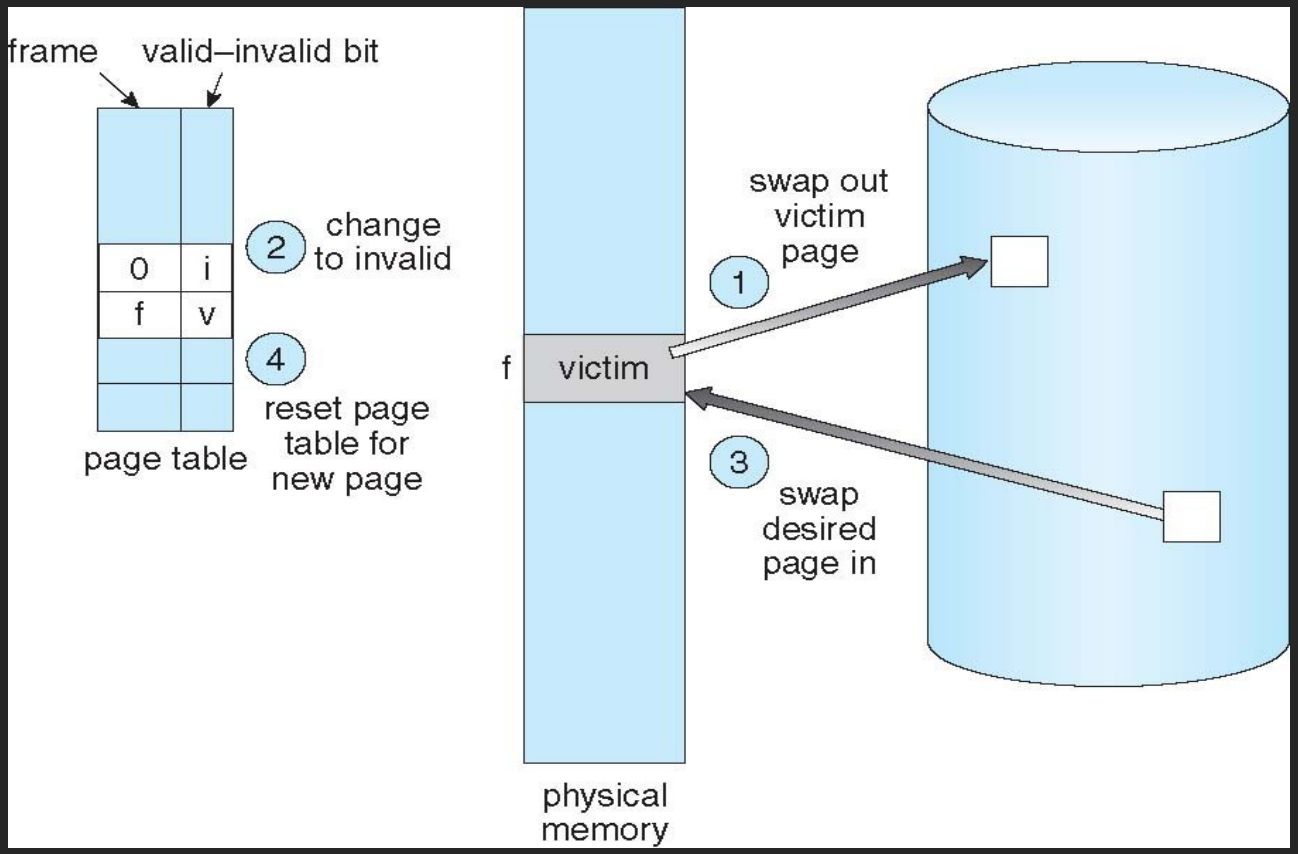


Substituição de páginas

- Esquema básico da substituição de páginas:
  1. Libera um quadro que não está sendo utilizado no momento
     - Grava quadro liberado no disco
     - Tabela de páginas deve ser atualizada
  2. Aloca quadro liberado para processo que gerou erro de página
     - Tabela de páginas deve ser atualizada
- Problema: são necessárias duas transferências de páginas
  - Uma do quadro a ser liberado para o espaço de troca, seguida de outra do espaço de troca para esse mesmo quadro
  - Substituição dobra o tempo de manipulação do erro de página

Substituição de páginas

- Esquema básico da substituição de páginas:



Substituição de páginas

- Bit de modificação (bit poluído/dirty bit): ajuda a diminuir overhead da substituição de páginas
  - Reduz tempo de E/S pela metade se não há modificação no quadro a ser liberado
- Funcionamento:
  - Cada página/quadro possui bit de modificação: inicialmente 0
  - Hardware seta automaticamente o bit para 1 quando algo é gravado na página
  - Quando um quadro for substituído, verifica-se o bit:
    - Se bit for 1: quadro é gravado no espaço de troca
    - Se bit for 0: quadro não é gravado

Substituição de páginas

- Rotina de serviço para tratar erros de página com suporte à substituição de páginas:
  1. Encontre posição da página desejada no disco
  2. Encontre um quadro livre
     a) Se houver quadro livre, utilize-o
     b) Caso contrário, use um algoritmo de substituição de páginas para escolher um quadro vítima
     c) Grave a vítima no disco se seu bit de modificação for 1
     d) Modifique as tabelas de páginas/quadros necessárias
  3. Leia a página desejada para o quadro livre e modifique as tabelas de páginas/quadros necessárias
  4. Reinicie o processo do usuário que gerou erro

Substituição de páginas

- Paginação por demanda + substituição de páginas:
  - Esquema funcional para execução de processos sem limitar espaço de endereçamento lógico ao tamanho da MP
    - Carrega somente o necessário
    - Se memória ficar lotada: substituição

Substituição de páginas

- Dois problemas a resolver:
  1. Qual a melhor" escolha de quadro para ser substituído?
     - Resposta: Algoritmo de substituição de páginas
       - Desempenho: deseja-se algoritmo com número mínimo de erros de páginas
  2. Quantos quadros devem ser alocados a cada processo?
     - Resposta: Algoritmo de alocação de quadros

Substituição de páginas

- Avaliação de algoritmos:
  - Utiliza-se sequência de referências à memória: sequência de referência
    - Pode ser aleatória ou através de monitoramento de um sistema
  - É medida a quantidade de erros de página por algoritmo

Substituição de páginas

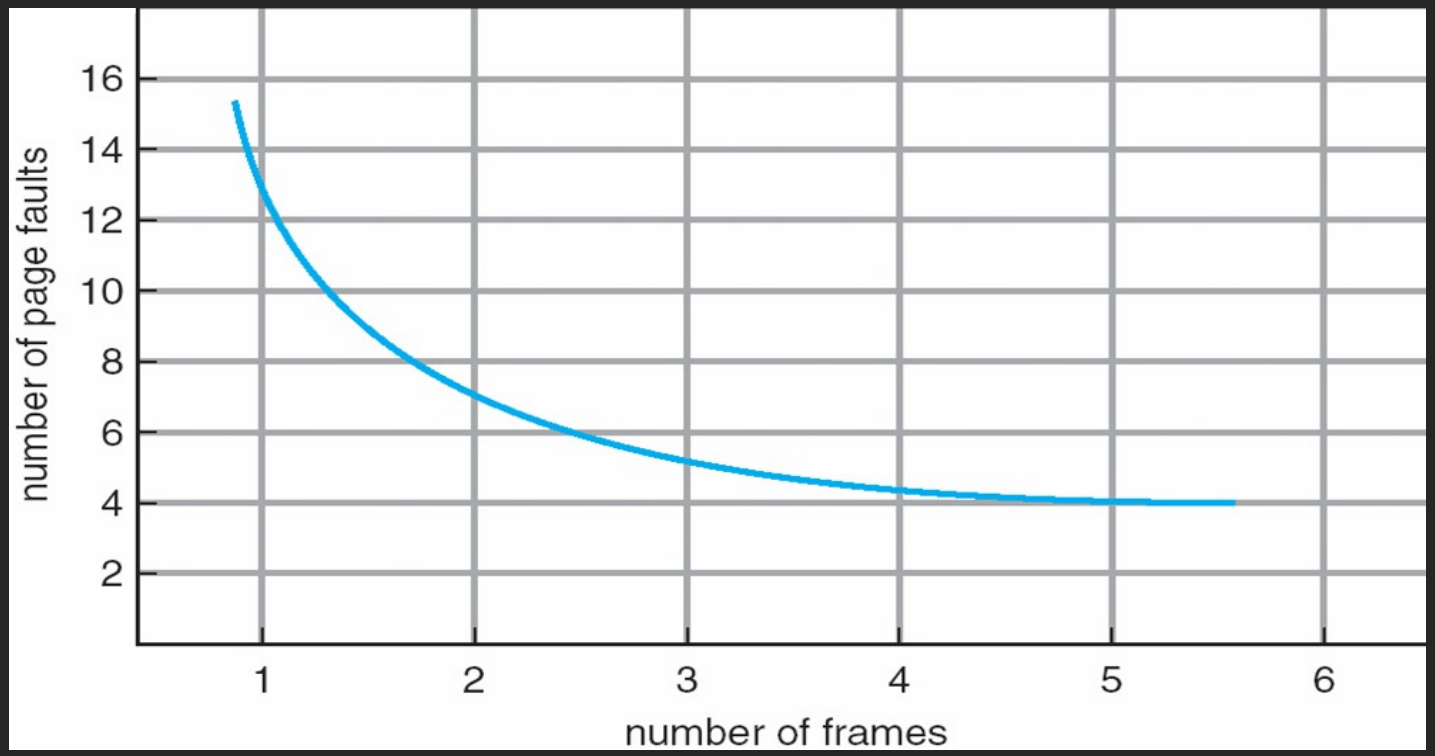
- Exemplo de monitoramento de processo (página de 100 bytes):
  - Sequência de endereços acessados:

0100, 0432, 0101, 0612, 0102, 0103, 0104, 0101, 0611, 0102, 0103,
0104, 0101, 0610, 0102, 0103, 0104, 0101, 0609, 0102, 0105

  - Sequência de referência: 1, 4, 1, 6, 1, 6, 1, 6, 1, 6, 1
- Cálculo do número de erros de página:
  - Qual algoritmo? Quantos quadros disponíveis?
    - Se houvessem 3 quadros ou mais: apenas 3 erros de página
    - Se houvesse 1 quadro: 11 erros de página

Substituição de páginas

- Curva esperada do número de faltas de página em relação ao número de quadros:

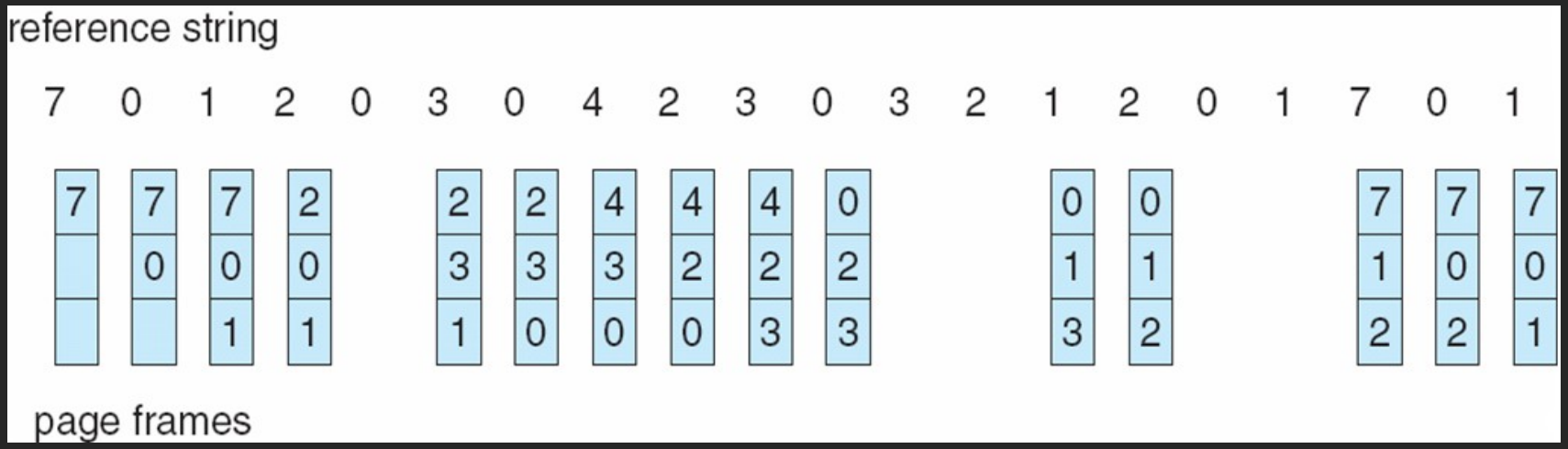


Substituição de páginas

- Substituição de páginas FIFO: mais simples
  - Associa a cada página o momento em que foi carregada na MP
  - Quando é necessário substituir, escolhe página mais antiga
  - Pode ser implementada como fila FIFO
  - Algoritmo simples e fácil de implementar
  - Desempenho pode não ser bom
    - Pode substituir página não muito utilizada: bom
    - Pode substituir página muito utilizada: ruim
    - Também pode expulsar página ativa: péssimo
      - Ela terá que voltar imediatamente à MP

Substituição de páginas

- Sequência de referência padrão e operação do algoritmo FIFO com 3 quadros:

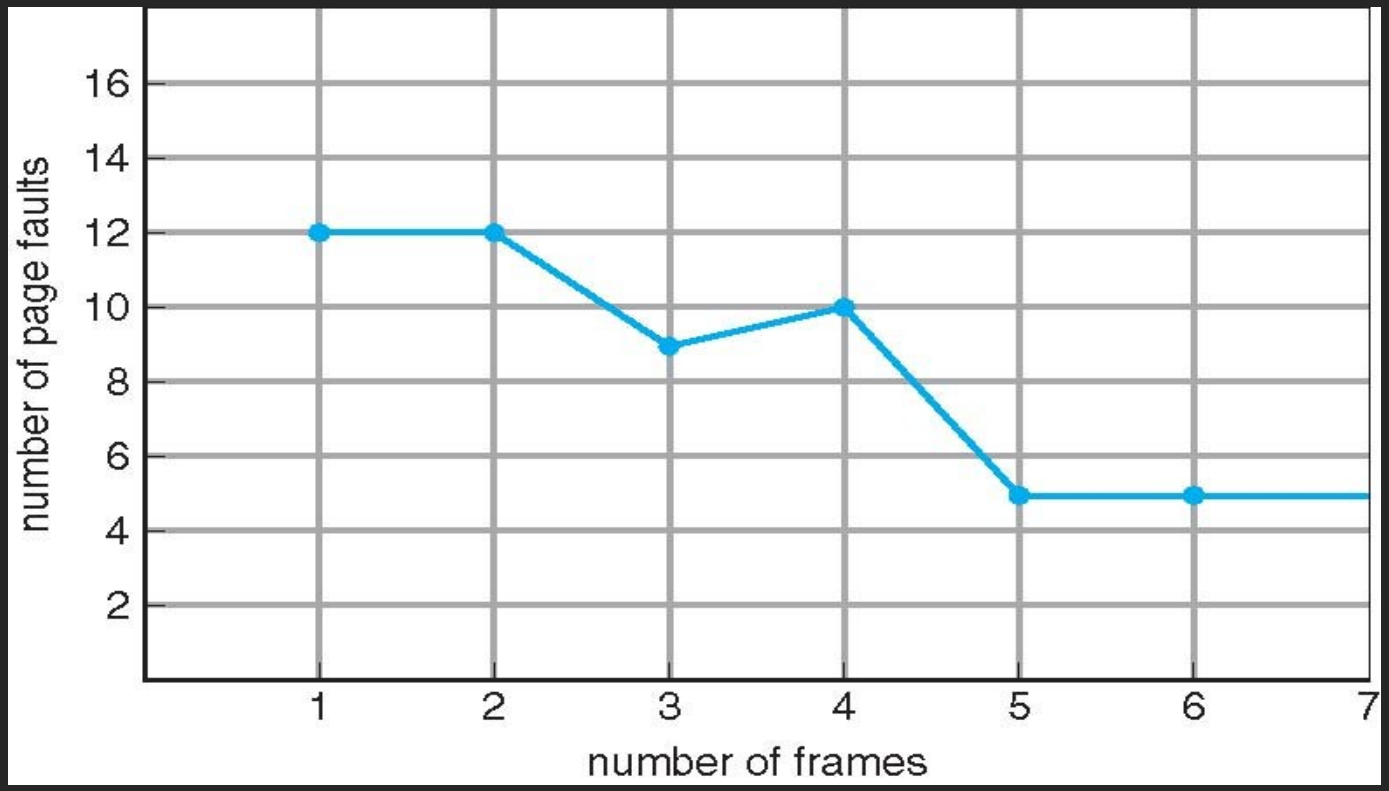


Substituição de páginas

- Anomalia de Belady: taxa de erro pode aumentar com aumento de número de quadros
  - Algoritmo FIFO e sequência: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
    - Número de erros com 3 quadros: 9
    - Número de erros com 4 quadros: 10
  - Pode acontecer com outros algoritmos

Substituição de páginas

- Ilustração da anomalia de Belady:

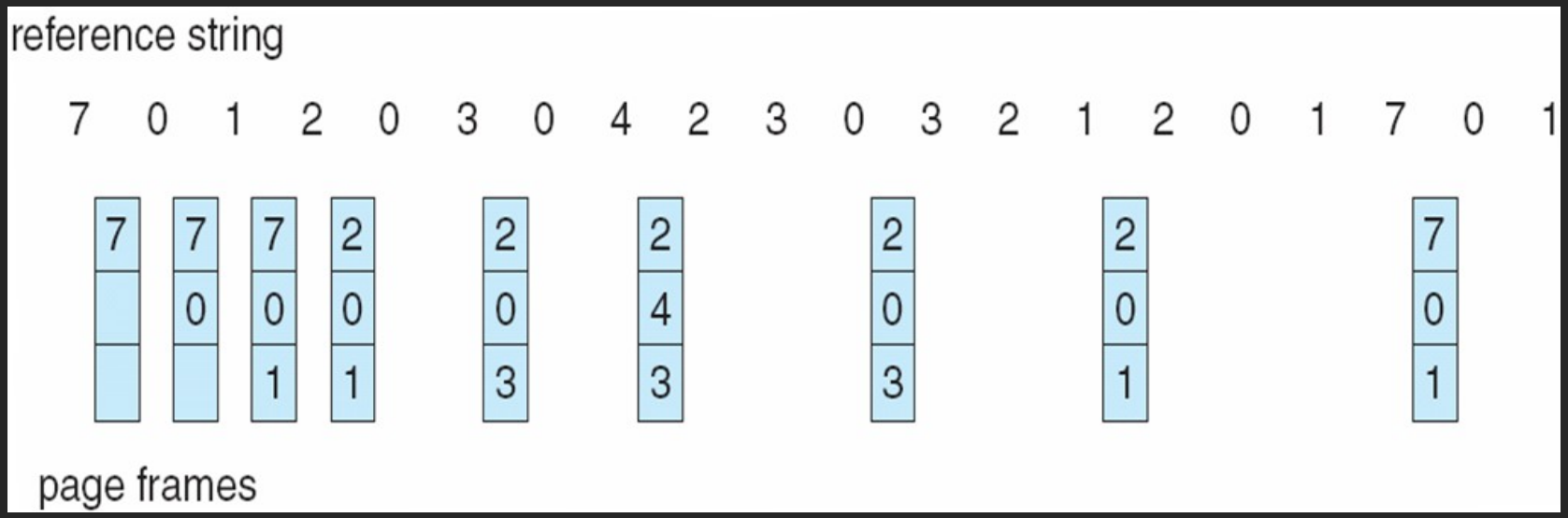


Substituição de páginas

- Algoritmo de substituição de páginas ótima (OPT)
  - Tem a taxa de erros mais baixa possível
  - Nunca é afetado pela anomalia de Belady
  - Algoritmo: “Substitua a página que não será utilizada pelo maior período de tempo”
  - Ex.: seqüência de referência padrão e três quadros
    - Gera 9 erros de página
    - FIFO gera 15

Substituição de páginas

- Sequência de referência padrão e operação do algoritmo OPT com 3 quadros:



Substituição de páginas

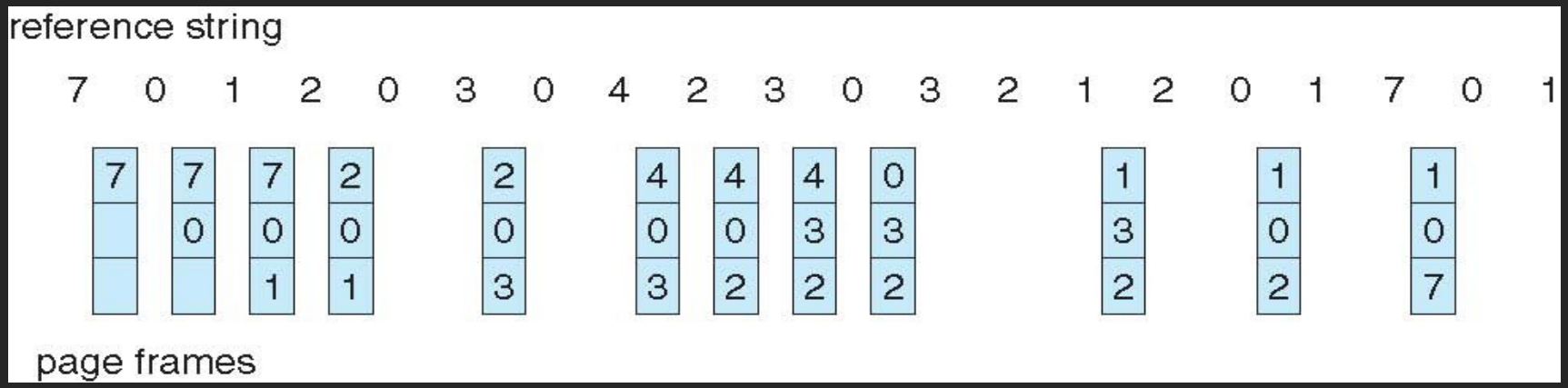
- Algoritmo OPT: idéia simples
  - Implementação: é preciso conhecer, a priori, sequência de referências
    - Futuro da execução do processo
  - É utilizado como parâmetro na avaliação de outros algoritmos

Substituição de páginas

- Substituição de páginas LRU (least recently used): menos recentemente usada
  - Substitui página que não foi usada por um longo período
    - Passado recente é usado como aproximação para o futuro (OPT)
  - Associa a cada página o momento da última utilização
    - Substitui página com tempo mais antigo
  - LRU também não sofre da anomalia de Belady
  - Ex. para sequência padrão e três páginas: 12 erros de páginas

Substituição de páginas

- Sequência de referência padrão e operação do algoritmo LRU com 3 quadros:



Substituição de páginas

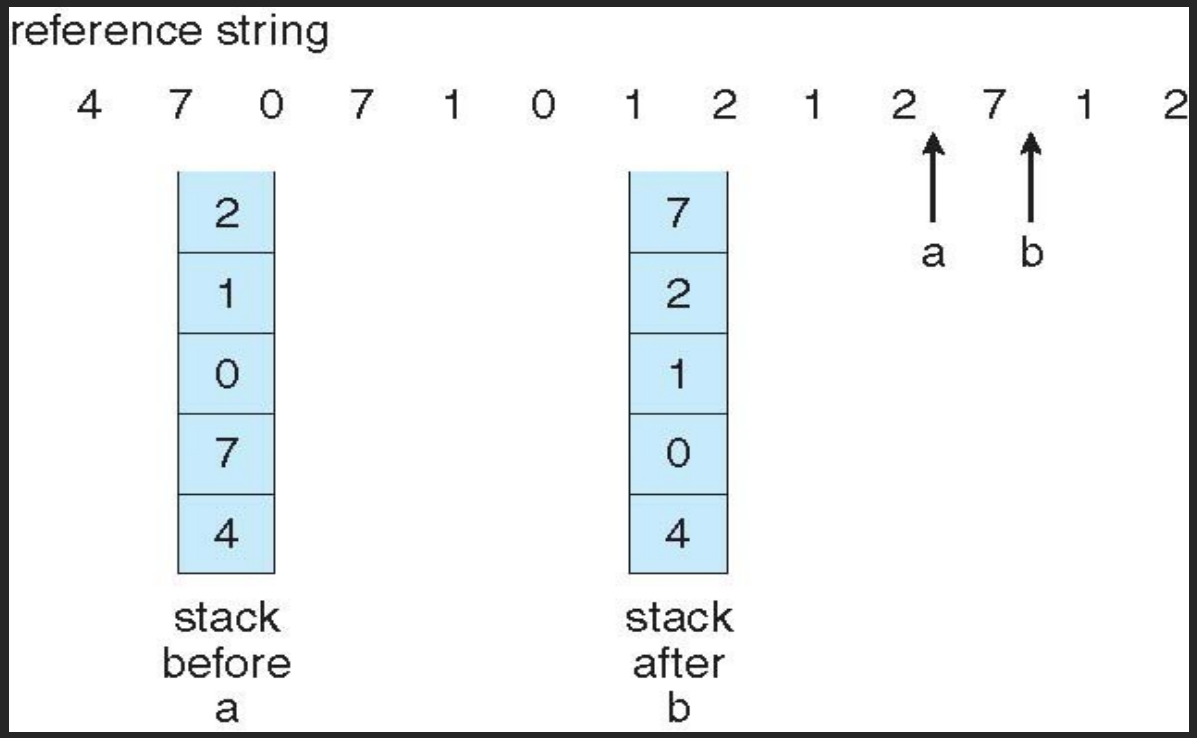
- Algoritmo LRU é frequentemente empregado
  - Implementação: requer substancial suporte de hardware
    - Implementação 1: contadores
      - Cada entrada da tabela possui campo de tempo-de-uso
      - Contador (relógio lógico) é incrementado a toda referência à MP
      - Valor do contador é sempre copiado para entrada da tabela de páginas quando se faz referência a essa entrada
      - É substituída a página que tem menor tempo-de-uso
        - É necessário buscar na(s) tabela(s) de páginas para encontrar a página LRU: tempo adicional
        - É necessário atualizar campo tempo-de-uso (na MP) a cada acesso à MP!!!

Substituição de páginas

- Implementação 2: pilha
  - Pilha mantém números de página
  - Quando página é referenciada, é movida para topo da pilha
    - Topo será a mais recentemente usada
    - Base será a página LRU
  - Remover de forma eficiente da pilha: lista duplamente encadeada
    - Mover para o topo pode precisar atualizar até 6 ponteiros: remoção (2), inserção (2), ptr base (1) e ptr topo (1)
  - Atualização mais cara (remoção com inserção)
  - Não existe necessidade de busca para substituição: ponteiro da base
- Implementação com contador ou pilha: inviáveis sem hardware adicional <u>além de registradores e TLB convencional</u>
  - Atualização da pilha ou campos da tabela para toda referência à MP

Substituição de páginas

- Uso de uma pilha para registrar páginas recentemente usadas:



Substituição de páginas

- Sistemas de computação podem:
  - Não oferecer suporte algum de hardware
    - Deve-se utilizar um algoritmo simples: FIFO
  - Oferecer suporte completo ao LRU:
    - Poucos suportam
  - Suportar parcialmente o LRU
    - É o que ocorre em geral

Substituição de páginas

- Algoritmos de aproximação ao LRU:
  - Em geral arquitetura oferece ajuda: bit de referência
    - Bit é ligado pelo hardware sempre que página é referenciada
    - Cada entrada da tabela possui bit de referência
  - Funcionamento
    - Inicialmente todos bits de referência são zerados pelo SO
    - Durante execução, bits são ligados pelo hardware
  - Ajuda dada pelos bits de referência
    - É possível saber as páginas que foram utilizadas e as que não foram
    - Não se sabe a ordem de utilização

Substituição de páginas

- Algoritmo dos bits de referência adicionais:
  - Cada tabela de página possui bits adicionais (8 bits, por exemplo)
  - De tempos em tempos (a cada 100ms, por exemplo) é programada uma interrupção
    - SO insere o bit de referência como bit de mais alta ordem em cada entrada, deslocando os demais bits para a direita
  - Esse algoritmo possibilita informação sobre ordenação
    - Em oito períodos de tempo:
      - Página que tiver bits 00000000: não foi acessada
      - Página que tiver bits 11000100 foi usada mais recentemente que 01110111

Substituição de páginas

- Algoritmo dos bits de referência adicionais (cont.):
  - A página com número mais baixo é “LRU” e será substituída
    - Pode haver mais de uma: substitui todas ou em ordem FIFO
  - Não é necessário atualização das páginas na MP a cada acesso
    - De tempos em tempos: mais eficiência
  - Número de bits adicionais pode variar
    - Em caso extremo há zero bits adicionais: só o de referência
      - Algoritmo de substituição de páginas da segunda chance

Substituição de páginas

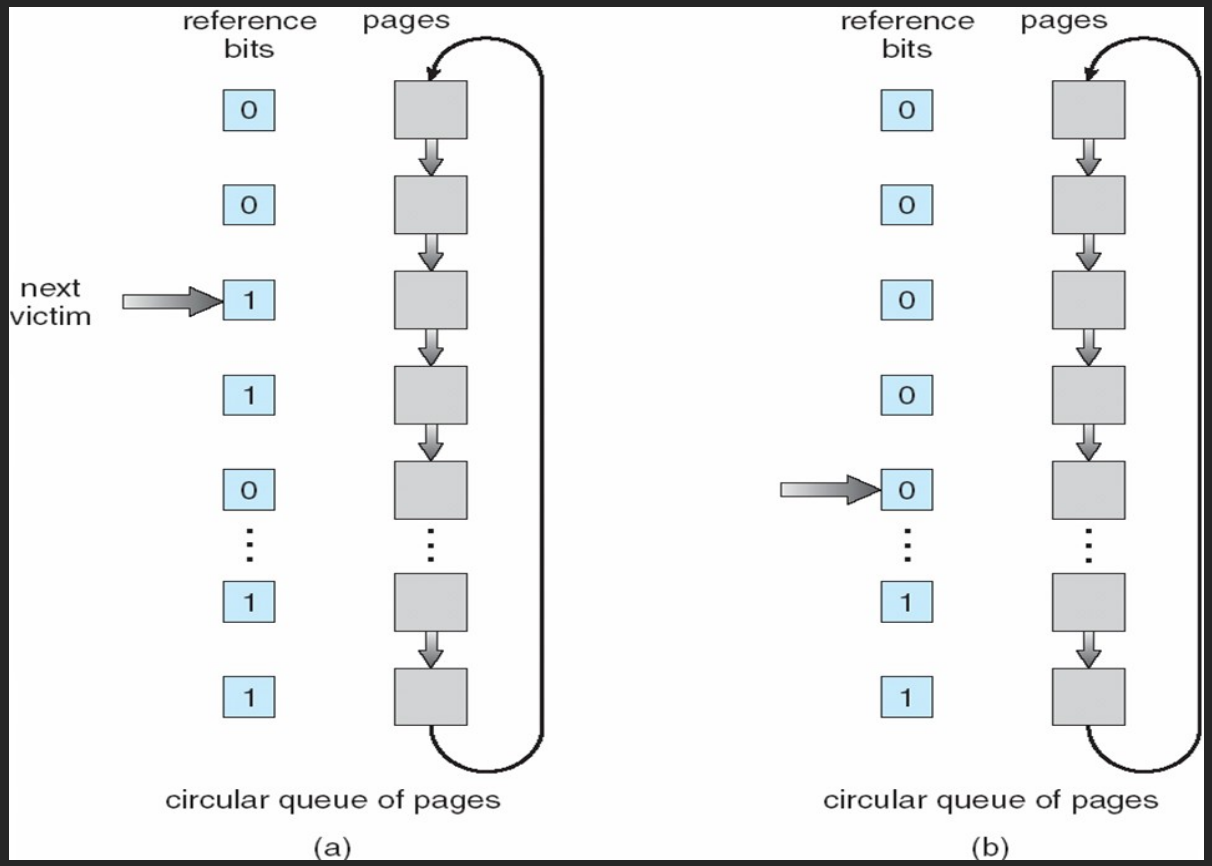
- Algoritmo da segunda chance:
  - Funcionamento FIFO:
    - Quando página é selecionada, verifica-se o bit de referência
      - Se for 0: página é substituída
      - Se for 1: página recebe segunda chance e substitui-se a próxima na ordem FIFO
    - Quando página recebe segunda chance
      - Bit de referencia passa a ser 0
      - Entra no final da fila
    - Se página é muito utilizada: não é substituída
      - Bit é trocado para 0, mas volta a ser 1 logo em seguida

Substituição de páginas

- Implementação do algoritmo da segunda chance: fila circular
  - Ponteiro indica qual página será substituída primeiro
  - Quando necessitar substituição:
    - Ponteiro avança até encontrar página com bit 0
    - Conforme avança, vai desligando bits de referência
      - Se todos os bits estão ligados: todas páginas recebem segunda chance e a primeira é substituída

Substituição de páginas

- Algoritmo da segunda chance:



Substituição de páginas

- Algoritmo da segunda chance estendido:
  - Usa dois bits: bit de referência e bit de modificação
  - Possibilidades:
    - (0, 0) não usada recentemente e não modificada
      - Melhor para substituir
    - (0, 1) não usada recentemente mas modificada
      - Não muito adequada porque necessita de gravação antes de substituir
    - (1, 0) recentemente usada e não modificada
      - Deve ser provável que será utilizada em breve novamente
    - (1, 1) recentemente usada e modificada
      - Provavelmente será usada novamente e terá que ser gravada em disco antes de ser substituída: pior para substituir

Substituição de páginas

- Algoritmo da segunda chance estendido:
  - Cada página fica em uma dessas classes
  - Substitui a primeira página encontrada na classe mais baixa
    - Pode ser necessário percorrer a fila circular várias vezes
      - Verificar cada classe

Substituição de páginas

- Outros algoritmos baseado em contagem de número de referências:
  - Mantém um contador de referências feito a cada página
  - LFU (menos frequentemente usada):
    - Substitui a página com menor valor do contador: assume que página não está sendo utilizada mais
  - MFU (mais frequentemente usada):
    - Substitui a página com maior valor do contador: assume que página com contador menor provavelmente acabou de ir para a memória e ainda não foi muito utilizada
  - LFU e MFU: implementações caras e não são boas aproximações de OPT

Substituição de páginas

- Além de algoritmo específico, outras técnicas são usadas para otimizar tempo de substituição
  - Reserva de cadeia de quadros livres:
    - Quando ocorre erro de página, vítima é selecionada, mas página desejada é lida antes que vítima seja gravada em disco
    - Processo aguardando página é reiniciado mais rapidamente
    - Quando vítima é gravada em disco, quadro livre é adicionado à cadeia
  - Expansão da ideia da cadeia de quadros livres
    - Manter lista de páginas modificadas
      - Sempre que dispositivo de paginação estiver ocioso:
        - Grava página modificada em disco
        - Bit de modificação passa a ser 0
      - Pode evitar necessidade de gravação futura

Substituição de páginas

- Outra expansão da ideia de cadeia de quadros livres:
  - Manter cadeia e saber qual página estava em qual quadro
  - Permite reutilizar página sem nenhum E/S, se for utilizada antes do seu quadro ser reutilizado
  - Funcionamento:
    - Quando ocorrer erro de página, verificar primeiro se a página está na cadeia de quadros livres
      - Se estiver: não necessita de E/S
      - Caso contrário: seleciona quadro livre e lê página

Alocação de quadros

- Número máximo de quadros: definido pela quantidade de memória física disponível
- Número mínimo de quadros: definido pela arquitetura
  - Quantidade mínima para executar todas as instruções possíveis
    - Ex. 1: instrução faz referência a um endereço de memória (ex: load endereco_direto)
      - Precisa de pelo menos 2 páginas: 1 para instrução + 1 para dado
    - Ex. 2: load endereço_indireto_um_nível
      - Precisa de pelo menos 3 páginas: 1 para instrução + 1 para operando que tem endereço + 1 para dado propriamente dito

Alocação de quadros

- Ex. 3 IBM 370: instrução MVC (movimentação de bytes)
  - Instrução pode ocupar 6 bytes: utilizar 2 páginas só para instrução (final página)
  - O bloco que bytes a ser movimentado pode ocupar 2 páginas (final página)
  - O destino dos bytes pode ocupar 2 páginas (final da página)
  - Instrução, como um todo: pode necessitar de 2+2+2 = 6 páginas
- Além disso, à medida que número de quadros por processo diminui, aumenta a taxa de erros de página
  - Perda de desempenho

Alocação de quadros

- Algoritmos de alocação:
  - Alocação igual: dados m quadros e n processos, cada processo recebe até m/n quadros
  - Alocação proporcional: Assume que processos diferentes precisam de quantidades diferentes
    - Ex. : Banco de dados ativo (127KB) + processo simples (10KB)
      - BD deveria receber mais memória que processo simples
        - Processo simples provavelmente não vai utilizar metade da memória
    - Cada processo recebe de acordo com seu tamanho

Alocação de quadros

- Alocação proporcional (cont.):
  - Considere Si o tamanho da memória virtual para o processo Pi
  - S = ∑ Si (memória virtual de todos os processos)
  - Se a quantidade de quadros disponíveis for m, cada processo receberá ai quadros, onde ai = (Si/S)*m
    - Entretanto, deve-se ajustar ai, de forma que seja pelo menos o mínimo requerido pelo conjunto de instruções, e o total não exceda m
  - Para o exemplo anterior: se o tamanho de página é 1kb e há 62 quadros livres, mas há instruções que precisam de pelo menos 6 páginas
    - a1 = (127/137)*62 = 57,4... Serão alocados 57
    - a2 = (10/137)*62 = 4,5... Serão alocados 4 (sobra 1 quadro livre)
      - Como 6 quadros são necessários, então uma alocação válida é: a2 = 6 e a1 = 56
  - Outra ideia: fazer proporcionalidade usando tamanho+prioridade

Alocação de quadros

- Alocação global versus alocação local:
  - Alocação global: utiliza substituição global de páginas
    - Um processo pode retirar páginas de outro da memória
      - Não permite controlar a taxa de erros de páginas por processo individualmente: número de quadros muda durante execução
      - Processos mais prioritários podem utilizar quadros de menos prioritários
      - Maior throughput
    - Método mais comum
  - Alocação local: utiliza substituição local de páginas
    - Um processo só pode substituir suas páginas
      - Permite controlar taxa de erros de páginas de processos individualmente
      - Páginas pouco utilizadas ficam retidas na memória: de outros processos

Atividade improdutiva

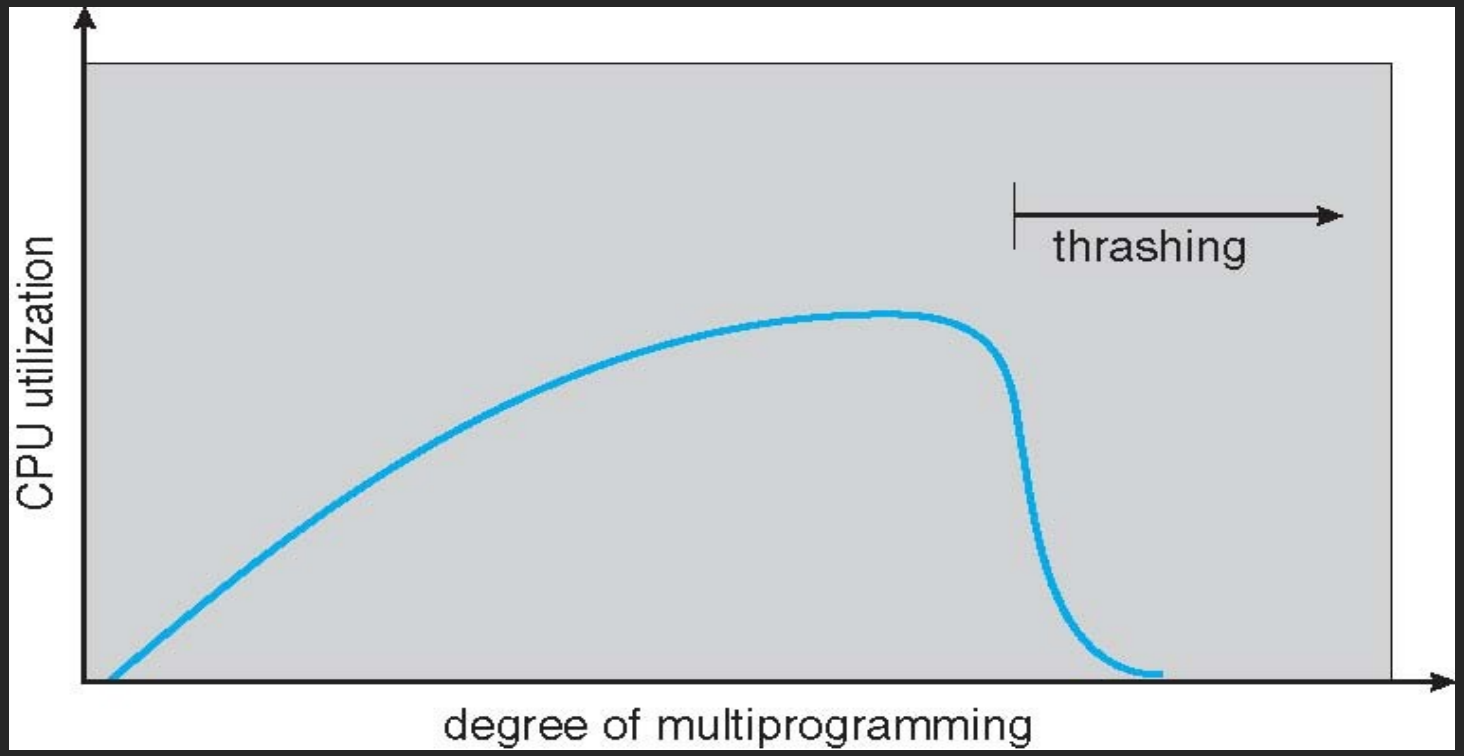
- Se processo possui poucos quadros:
  - Teoricamente: pode ser executado (até o mínimo da arquitetura)
    - Sua execução pode gerar muitos erros de páginas na prática
      - Páginas substituídas são muito usadas
  - Atividade improdutiva: processo gasta mais tempo paginando que executando

Atividade improdutiva

- Possível causa da atividade improdutiva:
  - O SO monitora uso da CPU:
    - Se o uso for muito baixo, insere novo processo: aumenta grau de multiprogramação
    - Há substituição de páginas global
    - Um processo começa a precisar de muitos quadros e toma quadros dos outros processos
    - Demais processos ficam com poucos quadros: esses processos têm muitos erros de página e a fila de prontos fica vazia
    - SO identifica baixa utilização da CPU e vai aumentando o grau de multiprogramação
    - Problema piora ainda mais ...

Atividade improdutiva

- Utilização da CPU vs. Grau de multiprogramação e atividade improdutiva



Atividade improdutiva

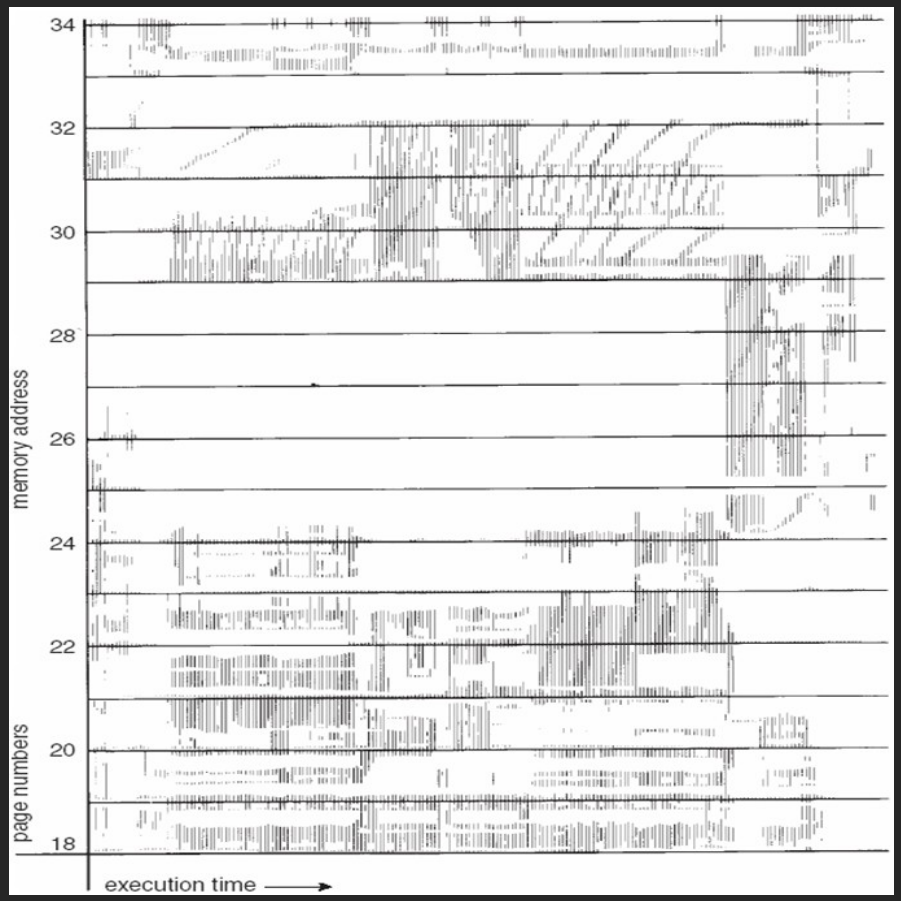
- Para limitar efeito da atividade improdutiva
  - Utilizar algoritmo de substituição de páginas local
- Para prevenir improdutividade em processo
  - É necessário fornecer quadros necessários
    - Como saber quais os necessários?
- Estratégia do conjunto ativo:
  - Verifica quantos quadros processo está realmente usando: modelo de localidade de execução do processo

Atividade improdutiva

- Modelo de localidade:
  - “Durante a execução, processo se moverá de localidade em localidade”
  - Localidade: conjunto de páginas ativas usadas juntas
  - Processo: composto por diversas localidades diferentes
    - Pode haver sobreposição
    - Ex.: sub-rotina de um processo define uma localidade
      - É acessado o código e dados locais da sub-rotina e também dados globais
      - Quando muda a sub-rotina, muda o código e dados locais (nova localidade)
  - Se alocar quadros suficientes para manter localidade corrente de processo:
    - Erros serão gerados somente até carregar as páginas da localidade

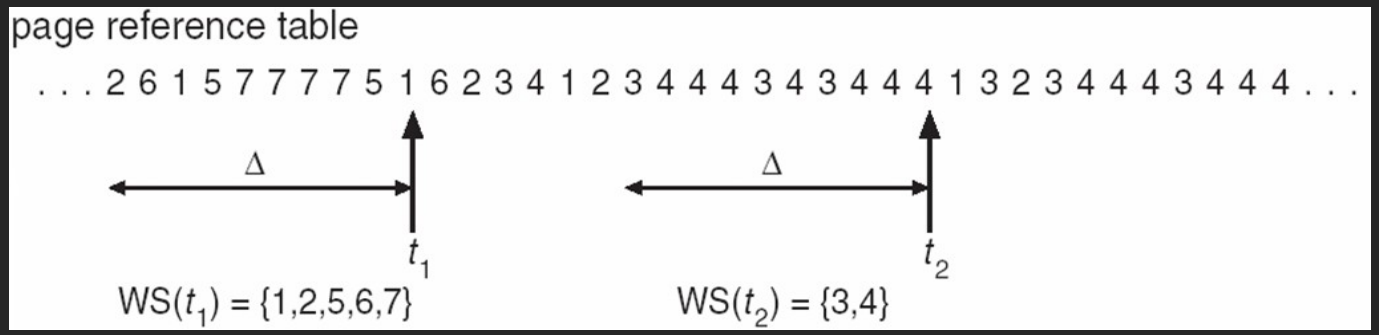
Atividade improdutiva

- Localidade em um padrão de acesso à memória:



Atividade improdutiva

- Modelo do conjunto ativo:
  - Baseia-se na idéia da localidade
  - Utiliza parâmetro Δ para definir janela do conjunto ativo
  - O conjunto ativo contém as Δ páginas mais recentemente referenciadas
  - Ex.: Δ = 10
    - Em t1 o conjunto ativo é {1, 2, 5, 6, 7}. Em t2 o conjunto é {3, 4}.



Atividade improdutiva

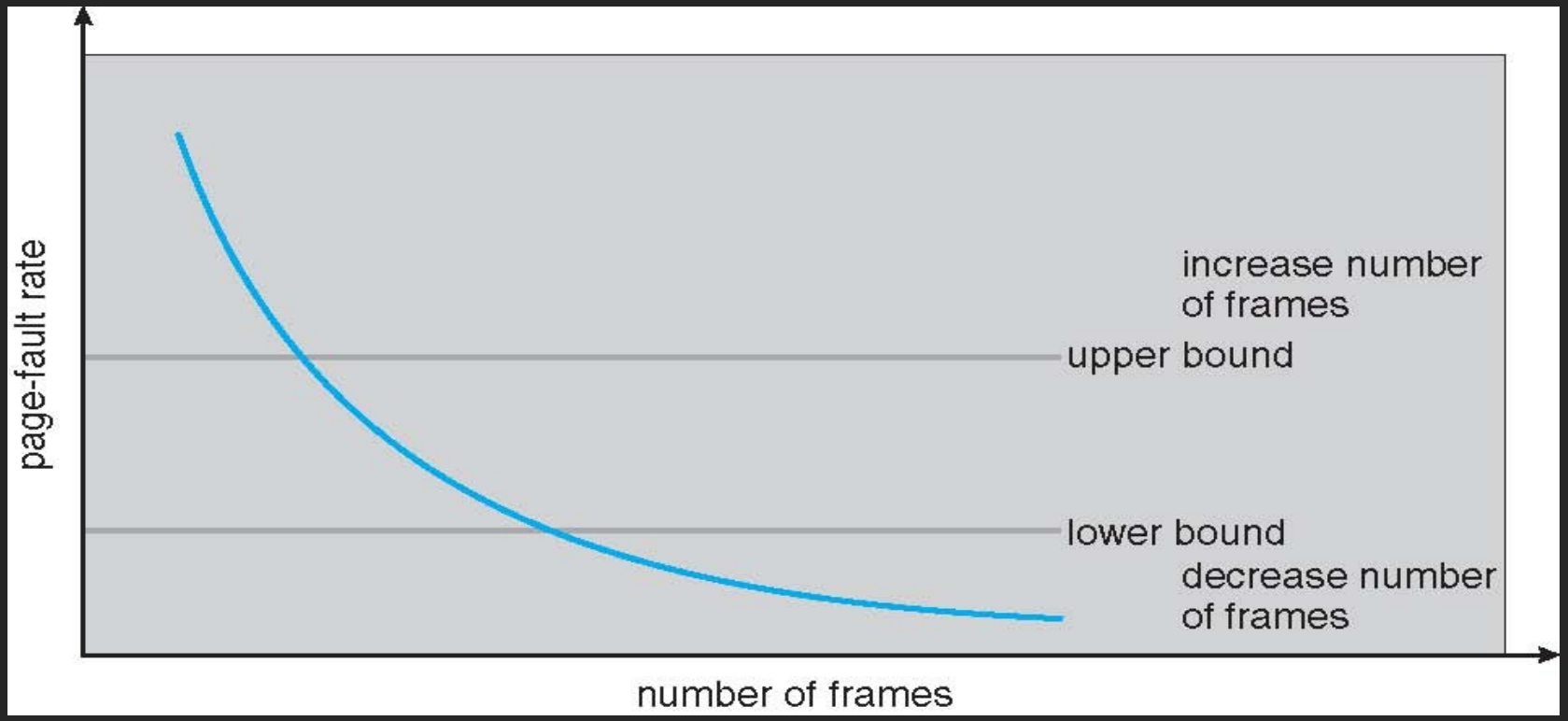
- Modelo do conjunto ativo:
  - Precisão do conjunto ativo depende de Δ
    - Δ muito pequeno: não abrangerá a localidade
    - Δ muito grande: haverá mistura de localidades
  - Propriedade importante: dado Δ e calculados o tamanho do conjunto ativos de cada processo i, WSSi
  - A demanda total por quadros será D = ∑ WSSi
  - Se quadros disponíveis < D: haverá atividade improdutiva
    - Alguns processos não terão quadros suficientes
    - Nesse caso, o SO deve suspender processo, liberando quadros
  - Dificuldade com o modelo do conjunto ativo
    - Administrar conjunto ativo: conjunto varia conforme a cada nova página acessada

Atividade improdutiva

- Frequência de erros de página (PFF, page-fault frequency):
  - Método mais simples que o do conjunto ativo
    - Atividade improdutiva: caracterizada por alta taxa de erros de página
    - Se taxa for muito baixa: processo tem quadros em excesso
  - Funcionamento:
    - É definido um limite inferior e um superior para a taxa de erro de página
      - Se taxa excede limite superior: outro quadro é alocado ao processo
        - Se não há quadro disponível: um processo será suspenso e terá seus quadros liberados
      - Se taxa cai abaixo do limite inferior: processo perde um quadro
  - Método possibilita controlar diretamente a taxa de erros e prevenir atividade improdutiva

Atividade improdutiva

- Ilustração do método PFF:

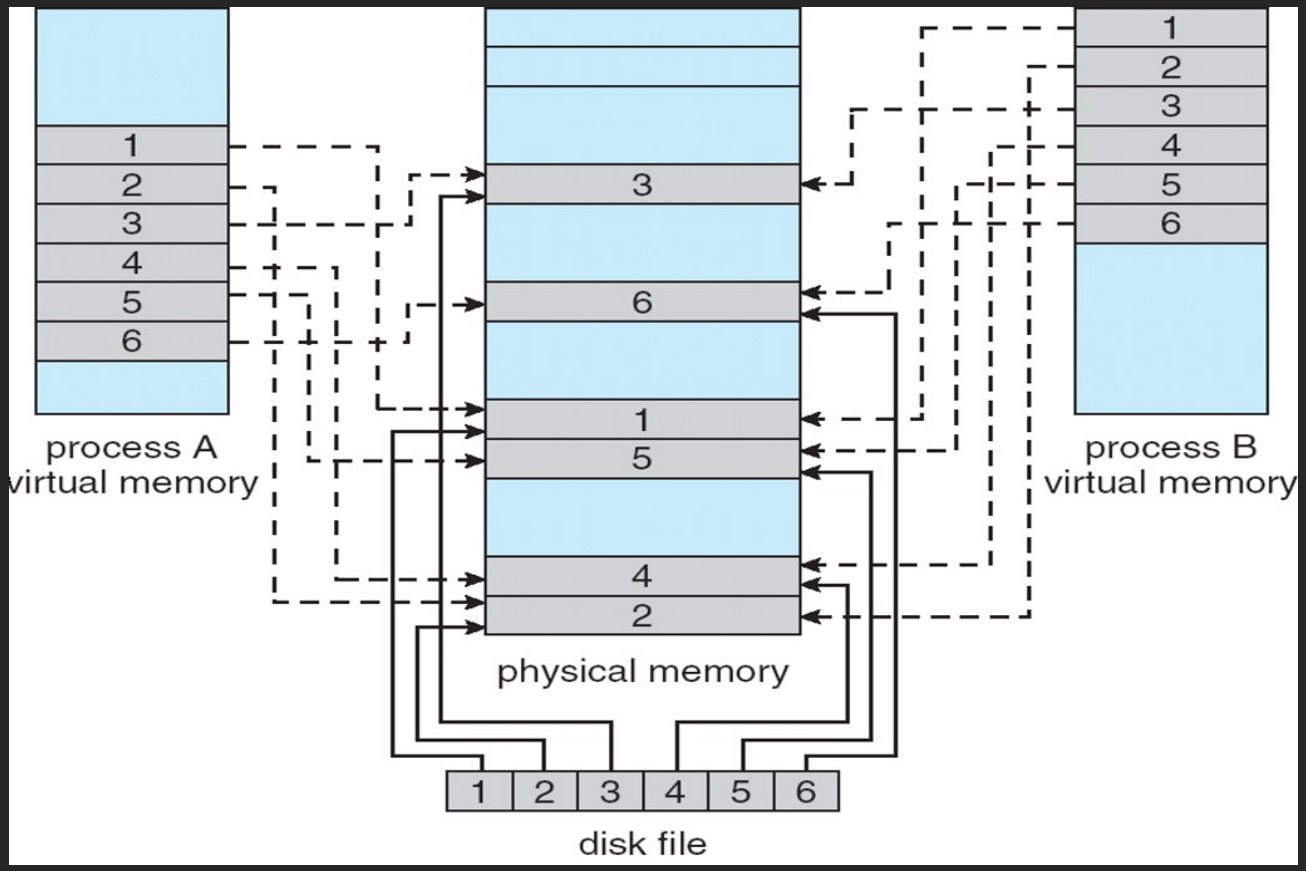


Arquivos mapeados em memória

- Acesso a arquivos através de chamadas padrão open(), read(), write()
  - A cada acesso: 1 chamada + 1 acesso a disco
- Acesso mais eficiente:
  - Mapear blocos de disco em página(s) da memória
  - Acesso inicial ao arquivo: ocorre com paginação por demanda
    - Resulta em erro de página: dados são carregados para memória
  - Leitura e escritas posteriores: feitas apenas com acesso à MP
    - Gravação em disco pode ocorrer somente no fechamento do arquivo, ou de tempos em tempos pelo SO

Arquivos mapeados em memória

- Alguns SOs necessitam chamada específica para fazer mapeamento
  - Outros, utilizam por default
- Processos podem:
  - Permitir que outros processos mapeiem um arquivo em sua memória: compartilhamento
    - Quando qualquer processo altera: todos vêem alteração
  - Chamadas ao sistema de mapeamento
    - Podem suportar cópia-após-gravação
    - Dessa forma: processo pode compartilhar arquivo read-only

Arquivos mapeados em memória

- Exemplo de arquivo mapeado em memória